

Atividade 5 - Módulo 5: Arquitetura Enterprise (fecha a disciplina)

Disciplina: Arquitetura de Sistemas com IA · Prof. Ahirton Lopes, Ph.D.

Missão Prática #05 - última da disciplina

Esta é a entrega que fecha a disciplina inteira: você vai estender o Gateway do Módulo 4 com uma cascata de model tiering (Módulo 5.4) e controle de orçamento por tenant (Módulo 5.1/5.2), aplicados a uma tarefa real do seu próprio contexto de trabalho.

Passo 1 - Defina seus tiers

Escolha dois modelos (ou dois tiers, se usar API paga) pro seu contexto: um barato, tentado primeiro sempre; um caro, que só entra se o barato não convencer. Documente o custo estimado de cada um.

- Tier 1 (barato) - tentado sempre, primeiro.
- Tier 2 (caro) - só se o Tier 1 não convencer.
- Tier 3 (frontier, opcional) - só se o Tier 2 também não convencer.

Não copie o limiar do TrialForge (0,75) - ele é específico do nomic-embed-text em português. Mesma disciplina do Módulo 4.5: medir, não assumir.

Passo 2 - Calibre o limiar de escalação com dado real

Teste o sinal de confiança que vai decidir a escalação (método FrugalGPT) contra pares reais do seu domínio: uma pergunta simples com resposta claramente certa, uma pergunta ambígua com resposta duvidosa, e uma pergunta fora do domínio conhecido. O limiar escolhido deve separar claramente os casos “bastou” dos casos “duvidoso”, com folga - não no ponto exato de corte (Módulo 4.2).

Passo 3 - Implemente orçamento por tenant

- Existe um identificador de tenant (estudo, cliente, time) em toda requisição?
- O orçamento é verificado ANTES da chamada ao modelo, não depois, reaproveitando a lição do Módulo 5.2 no seu próprio protótipo?
- Existe uma regra fixa (não sujeita à cascata) pra tarefas de erro caro e irreversível?
- A trilha de auditoria registra: tier usado, se escalou, gasto acumulado, limite do tenant?

Passo 4 - Rode o protótipo e documente os três comportamentos

Execute o Gateway na sua máquina (a entrega oficial é em JavaScript, conforme a ementa; a versão Python é só referência) e registre o que aconteceu nos três casos: um caso que o Tier 1 resolve sozinho sem escalar, um caso que a cascata escala pro Tier 2 (ou além), e um caso bloqueado por orçamento antes de qualquer chamada de modelo.

Passo 5 - Reflexão final

Com base no volume de escalação que você observou, responda em uma frase: que sinal indicaria que sua cascata de dois tiers precisaria virar três? Pode ser uma fração de requisições que ainda escala mesmo depois de calibrar o limiar direito, um padrão de pergunta que o Tier 2 também erra, ou um custo acumulado que passou do orçamento esperado. Você não precisa adicionar o terceiro tier agora, só reconhecer o sinal que justificaria fazer isso.

Entrega

Um único arquivo contendo: a cascata calibrada com a evidência que justifica o limiar escolhido (Passo 1-2), o orçamento por tenant implementado (Passo 3), o log da execução mostrando os três comportamentos (Passo 4), e a reflexão final (Passo 5).

Dica de execução

Pensa numa central de atendimento qualquer, fora do contexto de estudo clínico: perguntas de rotina (“qual o horário de funcionamento?”) o Tier 1 resolve de sobra; pedidos que exigem redação cuidadosa e sem margem de erro (uma resposta que vira compromisso contratual, por exemplo) merecem regra fixa pro Tier 2, igual a síntese de CSR no TrialForge. O exercício é achar esse equivalente no seu próprio contexto, não repetir o do TrialForge.

Ahirton Lopes · AI Architecture Toolkit - UNIPDS: Arquitetura de Sistemas com IA

Prof. Ahirton Lopes, Ph.D. - GDE AI, Microsoft MVP, Senior Manager